Remote Controller with Fail-Safe Update

Software Classification

Reference	TBD **[TBD-1]**
Issue / revision	Issue 1, Revision 0 — DRAFT
Date	10 October 2026
Author	Andrew Toma
Open items	2 TBD (App. A)

Document change record

Issue	Rev.	Date	Change
1	0	2026-10-10	First draft

1.0 Purpose and Scope

This document answers one question: how severe is the worst failure of each software item, and so how much rigour does each item need? It is the record of each software classification determination [1, Sec. 3.5.2, p. 25].

The items are the software of the System and of its enabling systems, as defined in the Concept of Operations [4, Sec. 1.2.2]. The System imitates an uncrewed spacecraft whose primary mission objectives depend on this software; a failure would cost those objectives. Every statement in this document was stated or accepted by the author on 10 October 2026.

2.0 Documents

[1] NASA, *NASA Software Engineering Requirements*, NPR 7150.2D.
[2] NASA, *Software Assurance and Software Safety Standard*, NASA-STD-8739.8B, 2022.
[3] NASA, *NASA Software Engineering Handbook*, NASA-HDBK-2203, Version D, page "SWE-020 – Software Classification".
[4] *Remote Controller with Fail-Safe Update — Concept of Operations*, Issue 1, Revision 0, Draft.

3.0 Classification

Each item takes the highest class whose definition applies [1, Sec. 3.5.1, p. 24].

Software item	Use	Class
Controller firmware	FLIGHT	B
Onboard computer software	FLIGHT	B
Ground software in the mission operations center	GROUND	B
Bench test tools	TEST	C
Firmware build tools	BUILD	D

Legend: FLIGHT = runs in the space segment; GROUND = runs in the ground segment; TEST = used only to test the System; BUILD = used only to build it. Classes as defined in [1, App. D, pp. 79–86].

3.1 Rationale

Controller firmware: Class B. Class B covers flight software "required to operate the vehicle or space asset ... such as commanding of the vehicle or asset" [1, App. D, p. 81]; its examples include "fault protection" [1, App. D, p. 81]. The controller runs commands and returns to working firmware on its own when new firmware fails.

Onboard computer software: Class B. The same definition applies. It passes every command and firmware image to the controller, watches it, and restores it when its firmware cannot run.

Ground software: Class B. Class B examples include "command and control ground systems" [1, App. D, p. 81]. The ground software prepares and checks every command and firmware image sent to the spacecraft.

Bench test tools: Class C. Class C covers "software used for the testing of space assets" [1, App. D, p. 82], and Class B excludes systems "used in testing Class B systems" in a development environment [1, App. D, p. 81].

Firmware build tools: Class D. Class D covers "ground software tools that support engineering development" [1, App. D, p. 83].

Why not Class A. Class A is software for human space flight [1, App. D, p. 79]. The imitated spacecraft carries no crew.

4.0 Safety-Critical Designation

Software is safety-critical when a hazard analysis traces a hazard to it [2, Sec. 4.2, p. 20]; the determination is made for each component [1, Sec. 3.7.1, p. 26]. The designation of each item above is set after the hazard analysis **[TBD-2]**.

5.0 Tailoring and Review

- Software engineering and software assurance are expected to agree on each classification [1, Sec. 3.5.1 Note, p. 25]. One engineer builds the System, with no independent assurance team (ConOps constraint C-3), so the classification is the author's alone.
- The classification is revisited at each later phase [3, Sec. 3.5.1]. If a class changes, the plans are updated to match [1, Sec. 2.2.8, p. 19].

A Open Items

The values below are not yet known.

TBD-1 Project name and document reference.

TBD-2 Safety-critical designation of each software item, from the hazard analysis.